
Games Engineering Report

Jade(Yujie) Liang
5709365

University of Warwick

This project is a top-down 2D twin-stick shooter. The player controls an aircraft, fires projectiles at enemies, avoids dangerous terrain, and wins by surviving for the required time. The game uses the GamesEngineeringBase framework provided by the module. Most game logic is written in C++, while numerical values and image resources are read from text configuration files.

1 Game Introduction and Full Game Flow

When the game starts, it first loads Resources/gameplay.txt, Resources/Sprites/sprites.txt, Resources/tiles.txt, and the image files. If any file is missing or has an incorrect format, the game shows a resource error.

After the resources are loaded successfully, the player presses Enter to start Level 1. Level 1 uses a fixed 42 by 42 map. Each tile is 32 by 32 pixels, so the full map size is 1344 by 1344 pixels, which is larger than the 1024 by 768 window. The player can move with WASD or the left stick. The game automatically fires normal bullets at the nearest enemy. The player can also use the left mouse button or the right stick to fire an eight-direction projectile, and press Space to use the AOE attack.

Enemies keep spawning, and the spawn interval becomes shorter over time. Normal enemies directly chase the player, while turrets stay still and fire enemy projectiles. After killing enemies, the player can receive an attack-speed upgrade or an AOE target-count upgrade. Water on the map cannot be crossed, roads increase movement speed, lava can be crossed but continuously reduces health, and medkits restore health.

After 120 seconds in Level 1, the game does not pause. A portal appears near the centre of the finite

map. When the player touches the portal, Level 2 starts. Level 2 uses an infinite map generated from a random seed, and the camera always follows the player. When Level 2 reaches 120 seconds, the player wins and the screen shows the final score and an A, S, SS, or SSS grade. If health reaches zero in either level, the game shows the lose screen.

The main loop runs in this order: read input, calculate the time difference, update the game, draw the frame, and finally present it to the window.

```
1 while (running) {  
2     canvas.checkInput();  
3     const float dt = timer.dt();  
4     gameUpdate(dt);  
5     if (!running) break;  
6     paintAll();  
7     canvas.present();  
8 }
```

2 Technique One: Fixed Scene, Player Movement, and Virtual Camera

2.1 Principle

World coordinates and screen coordinates need to be separated. World coordinates show the real position of an object in the whole map, while screen coordinates show where it appears in the current window. If world coordinates are used directly for drawing, the player will disappear after moving outside the window, and a map larger than the window cannot be displayed correctly.

A virtual camera can be understood as a window moving over a large map. Assume the world position of an object is P_w and the world position of the camera top-left corner is C . Then the screen position P_s is:

$$P_s = P_w - C$$

When the camera follows the player, it can first be placed half a window size to the upper-left of the player:

```
cameraX = playerX - windowWidth/2,
cameraY = playerY - windowHeight/2
```

A finite map also needs to limit the camera position. The camera cannot move outside the map, otherwise black areas with no map content will appear on screen. An infinite map has no fixed boundary, so the camera can keep the player at the centre.

Player movement uses a two-dimensional vector. When two keyboard directions are pressed at the same time, the vector length becomes greater than 1. If this is not handled, diagonal movement will be faster than horizontal movement. Therefore, the input is normalized when its length is greater than 1. If the stick input is below 1, its original magnitude is kept, so lightly pushing the stick gives slower movement.

2.2 Project Implementation

Player stores the current position, previous-frame position, velocity, health, collision radius, and attack data. The position is updated as follows:

```
1 const float length = std::sqrt(input.x * input.x
  + input.y * input.y);
2 if (length > 1.0f) {
3     input.x /= length;
4     input.y /= length;
5 }
6 velocity = {input.x * speed * terrainMultiplier,
7             input.y * speed * terrainMultiplier
8             };
8 position.x += velocity.x * dt;
9 position.y += velocity.y * dt;
```

terrainMultiplier is 1 by default. When the player centre is on a road, it becomes 1.5, so the base speed of 180 pixels per second increases to 270 pixels per second.

Camera::follow calculates the camera position from the player position. In fixed mode, the camera is clamped to the map range. In infinite mode, it is not clamped.

```
1 position = {target.x - viewport.x * 0.5f,
2             target.y - viewport.y * 0.5f};
3 if (mode == CameraMode::Fixed) {
4     position.x = ClampValue(position.x, 0.0f,
5                             maxX);
6     position.y = ClampValue(position.y, 0.0f,
7                             maxY);
8 }
```

Before drawing the player, enemies, projectiles, and portal, the code always calls worldToScreen. In this way, movement and collision only work with world coordinates, and only rendering needs to care about the camera.

Updating the player and camera only needs a fixed number of calculations each frame, so the time complexity is $O(1)$.

3 Technique Two: NPC Spawning, Types, and Behaviour

3.1 Principle

If every enemy type has a completely separate code path, a large number of NPCs will create a lot of repeated code. A simpler method is to let enemies share one data structure and one update loop, then use enemy type and configuration data to decide health, speed, collision radius, damage, and sprite.

The most basic way to chase the player is to calculate the direction vector of "player position minus enemy position":

$$\text{direction} = \text{playerPosition} - \text{enemyPosition}$$

Dividing this direction by its length gives a unit direction. Multiplying it by enemy speed and dt moves the enemy directly toward the player. This method does not use pathfinding. The advantage is simple and low cost.

Random spawning needs to handle both inside-screen and outside-screen cases. For an inside-screen spawn, a random point can be selected inside the camera rectangle. For an outside-screen spawn, the code first randomly chooses the top, bottom, left, or right edge, then places the enemy a small distance outside that edge. The spawn point also needs to keep a safe distance from the player, so an enemy does not appear directly on top of the player.

3.2 Project Implementation

The project uses a fixed-length Enemy array that can store up to 256 enemies. A fixed array avoids dynamic memory allocation every frame and also makes it easy to save data in a fixed order. When an enemy is spawned, the code finds the first empty slot in the array. When an enemy dies or moves too far from the player, that slot can be reused.

The current properties of the four enemy types are: The enemy spawn interval has four stages:

Current Level Time	Spawn Interval
0 to 30 seconds	1.60 seconds
30 to 60 seconds	1.20 seconds
60 to 90 seconds	0.85 seconds
After 90 seconds	0.55 seconds

This meets the requirement that enemy frequency increases over time. Each spawn has a 30% chance to be inside the screen, while the remaining cases are outside the screen. The code tries at most 32 times to

Type	Appearance	Health	Speed	Contact Damage	Behaviour
Goblin	Orc animation	35	85	10	Normal chasing
Sprinter	Bat animation	18	145	7	Fastest speed but lower health
Brute	Fire stone statue animation	120	48	20	High health and damage but slow movement
Turret	Turret body and rotating barrel	65	0	12	Stays still and fires every 1.8 seconds

find a safe position. If no safe position is found, that spawn is skipped, which prevents the spawn loop from getting stuck.

The core movement code for normal enemies is:

```

1  const Vector2 delta = {
2      session.player.position.x - enemy.position.x
3      ,
4      session.player.position.y - enemy.position.y
5  };
6  const float distance = std::sqrt(delta.x * delta.x + delta.y * delta.y);
7  if (distance > 0 && stats.speed > 0) {
8      const float travel = ClampValue(stats.speed * dt, 0, distance);
9      enemy.position.x += delta.x / distance * travel;
10     enemy.position.y += delta.y / distance * travel;
11 }

```

The turret speed is set to zero, so it does not enter the movement calculation. Each turret stores its own firing cooldown. When the cooldown ends, it calculates the direction to the player and creates an enemy projectile. The barrel drawing angle uses `atan2`, so the turret body stays still while the barrel turns toward the player.

Enemy updates need to iterate through the whole fixed array, so the complexity is $O(E)$, where E is the enemy capacity. Finding the nearest enemy is also $O(E)$.

4 Technique Three: Collision System

4.1 Principle

Characters and projectiles in the game can be approximated as circles. To test whether two stationary circles collide, it is enough to compare the squared distance between their centres with the squared sum of their radii:

$$\text{distance}^2 \leq (\text{radiusA} + \text{radiusB})^2$$

Checking only the current-frame position causes a problem. If a projectile moves very fast, it may be on the left side of an enemy in the previous frame and already on the right side in the next frame. It passes through the enemy between the two frames, but neither endpoint overlaps. This problem is usually called tunnelling.

This project uses continuous circle sweep collision detection. The relative movement of two objects in one frame is changed into "a line segment moving toward a circle", and then the quadratic equation for the segment and circle is solved. If the solution is between 0 and 1, a collision happened during that frame. The returned fraction can also be used to decide which enemy a projectile hits first.

Terrain collision uses a circular player against square tiles. The code first finds the point on the water-tile rectangle that is closest to the player centre, then compares the distance from that point to the centre. If the distance is smaller than the player radius, the player is colliding with water.

4.2 Project Implementation

The main calculation in the continuous collision function is:

```
1 const Vector2 offset = {from.x - targetFrom.x,
    from.y - targetFrom.y};
2 const Vector2 motion = {
3     to.x - from.x - (targetTo.x - targetFrom.x),
4     to.y - from.y - (targetTo.y - targetFrom.y)
5 };
6 const float c = offset.x * offset.x + offset.y *
    offset.y - radius * radius;
7 const float a = motion.x * motion.x + motion.y *
    motion.y;
8 const float b = offset.x * motion.x + offset.y *
    motion.y;
9 const float discriminant = b * b - a * c;
10 fraction = (-b - std::sqrt(discriminant)) / a;
```

The same CircleSweep function is used for the following cases:

1. Player contact with enemies.
2. Normal player projectiles against enemies.
3. Eight-direction player projectiles against enemies.
4. Turret projectiles against the player.
5. The player collecting upgrades and medkits.
6. The player entering the portal.

After taking normal damage, the player has 0.35 seconds of invincibility. This avoids losing a full amount of health every frame while overlapping with an enemy. Lava causes continuous damage, so health is reduced by damage-per-second multiplied by dt, and it also triggers the player damage-flash effect.

Water collision during player movement is handled separately on the X and Y axes. Each movement is also split into small steps no longer than one quarter of a tile. This makes it less likely that the player crosses directly over water when movement speed increases, and it also allows the player to slide along the water edge.

Enemy contact with the player and enemy-projectile collision checks are $O(E + P_e)$. For the current player projectiles, each projectile checks all enemies, so the complexity is:

$$O((\text{automaticProjectiles} + \text{manualProjectiles}) \times \text{enemies})$$

Here, P_a is the number of automatic projectiles and P_m is the number of eight-direction projectiles. This implementation is direct and easy to show as correct, but it becomes a performance bottleneck when there are many enemies and projectiles at the same time.

5 Technique Four: Eight-Direction Attack, AOE, and Upgrades

5.1 Principle

The eight-direction attack needs to convert a continuous stick direction into one of eight fixed directions. A full circle is divided equally into eight parts, so each sector has an angle of $\pi/4$. First, $\text{atan2}(y, x)$ gets the input angle. Half a sector is then added as a rounding offset, and the result is divided by $\pi/4$ to get the sector number.

The AOE attack in this game targets the N enemies with the highest current health. The implementation repeatedly searches for the highest-health enemy that has not been selected yet. All targets are selected before damage is applied. This prevents an earlier target from dying and freeing an array slot before the later targets are selected.

5.2 Project Implementation

The project reads both controller-stick and mouse input. When the stick has input, the stick direction is used first. If there is no stick input and the left mouse button is held, the direction is calculated from the mouse position minus the player screen position.

```
1 const float angle = std::atan2(y, x);
2 int sector = int(std::floor((angle + 0.392699082
    f) / 0.785398163f));
```

```

3 if (sector < 0) sector += 8;
4 outWay = directions[sector % 8];

```

The eight-direction projectile currently deals 25 damage and has a base firing interval of 0.8 seconds. The automatic attack finds the nearest active enemy. Its normal projectile currently deals 12 damage and has a base interval of 0.5 seconds. Both linear attacks are affected by the attack-speed upgrade.

The AOE is triggered with Space. Its base cooldown is 12 seconds, each target takes 40 damage, and it initially selects 3 enemies. The selection process is:

```

1 while (targetCount < requested && targetCount <
  GameConfig::MaxAoeTargets) {
2   int best = -1;
3   for (unsigned int i = 0; i < GameConfig::
    MaxEnemies; ++i) {
4     const Enemy& enemy = enemies.at(i);
5     if (!enemy.active || selected[i])
      continue;
6     if (best < 0 || enemy.health > enemies.
      at(best).health) best = int(i);
7   }
8   if (best < 0) break;
9   selected[best] = true;
10  indices[targetCount++] = static_cast<
    unsigned int>(best);
11 }

```

One upgrade drops after every 5 enemy kills. Attack-speed upgrades and AOE target-count upgrades appear alternately. An attack-speed upgrade multiplies the current attack interval by 0.85, with a minimum interval of 0.08 seconds. Each AOE upgrade adds 1 target, with a maximum of 256.

In the worst case, AOE selection scans the enemy array N times, so the complexity is $O(NE)$. The normal nearest-target search is $O(E)$.

6 Technique Five: 32-Pixel Tiles, Fixed Map, and Infinite Map

6.1 Principle

A tile map divides the map into equal-sized cells, and each cell only stores a tile ID. During drawing, the corresponding image is selected from that ID. If the map width is W and the height is H , and each cell stores one byte, the map data only needs about WH bytes. There is no need to store all $32W \times 32H$ full pixels.

Data-driven design means the map layout is not hard-coded in C++. The program only reads the file format, while the real map tile IDs are stored in a text file. This means the map can be changed without changing the code.

An infinite map cannot store every tile in memory because its coordinates have no boundary. This project uses deterministic generation based on "coordinate plus seed". The same seed and the same coordinate always produce the same result. Therefore, when the player leaves an area and later comes back, the terrain

does not change, and the project does not need more and more map text files.

When roads connect across regions, neighbouring regions must use the same boundary data. Water and lava areas can be built from several ellipses, with some edge parts removed to make an irregular outline. Finally, the shoreline tile is selected according to whether the neighbours above, below, left, and right are water.

6.2 Project Implementation

The fixed-map file header stores the width, height, tile width, tile height, and layer number. It is followed by 42 by 42 tile IDs. The loader checks the amount of data, commas, value ranges, and extra data, and loads 0.png through 24.png.

```

1 tileswide 42
2 tileshigh 42
3 tilewidth 32
4 tileheight 32
5 layer 0
6 0,0,0,7,13,13,4,...

```

When drawing the visible area, the code first converts the camera range into tile rows and columns. It only iterates through the cells covered by the screen instead of iterating through the whole fixed map. The tile pixels for each cell are then written into the window buffer.

The current terrain types are:

Terrain	Effect
Grass	Normal movement
Road	Player speed becomes 1.5 times the base speed
Water and shoreline	Player cannot pass; enemies can pass
Lava	Passable, but deals 6 damage per second

The infinite map is divided into regions of 64 by 64 tiles. Coordinate hashing decides road turning points, pool centres, pool radii, lava areas, and short side roads. It uses the same 25 tile images, but the layout is generated from a seed, so Level 2 is different in each new game.

The infinite-map query does not store the whole world. A single coordinate query uses a fixed amount of calculation, so it can be treated as $O(1)$. The map work per frame depends on the number of visible tiles and the number of window pixels, which is $O(V)$, where V is the map area covered by the screen. The current implementation does not cache generated results, so the same visible tiles are calculated again in later frames. This is also a part that can continue to be optimized later.



Figure 2 shows the infinite map in Level 2. The screen contains crossing main roads, short dead-end roads, irregular lava, water, upgrades, and medkits. The HUD also shows the player coordinates, terrain speed, remaining time, FPS, kill count, and random seed.

7 Technique Six: Two-Minute Levels, Two Maps, FPS, and Score

7.1 Principle

Level timing should use game simulation time. Simulation time does not increase while the game is paused, so pausing for one minute does not allow the player to complete the level directly. The level transition can be treated as a simple state flow: Level 1 running, Level 1 portal open, Level 2 running, then victory or defeat. This project includes kills, number of upgrades, and remaining health in the final score.

7.2 Project Implementation

After 120 seconds in Level 1, LevelFlow sets portalOpen to true. The game keeps running after the portal appears, so the player can still move, shoot, and take damage. The code uses swept-circle collision to check whether the player enters the portal, so quickly passing through the portal can still trigger it.

When Level 2 starts, the current player health, upgrades, and attack cooldowns are kept. Level 1 enemies, projectiles, and dropped items are cleared. A new random map seed is then set and the level timer is reset to zero. When Level 2 reaches 120 seconds, the state changes to Victory.

The final score formula is:

$$\text{Score} = \text{Kills} \times 10 + \text{Bufs} \times 100 + \text{HP} \times 5$$

The grading rules are:

Score	Grade
Below 1500	A
1500 to 2999	S
3000 to 4999	SS
5000 or above	SSS

```

1 unsigned int Game::countPoint() const {
2     return combat.kills * 10 + combat.
      upgradesCollected * 100
3     + static_cast<unsigned int>(session.
      player.health + 0.5f) * 5;
4 }

```

FPS is calculated once every two seconds to reduce the value jumping every frame. FPS, level time, and score-related data are shown in the HUD, and the final screen shows the score again. The victory screen shows the grade, while the defeat screen does not.



Figure 3 shows the defeat screen after the player health reaches zero. It shows the score, kill count, number of upgrades, and remaining health, but it does not show a grade.

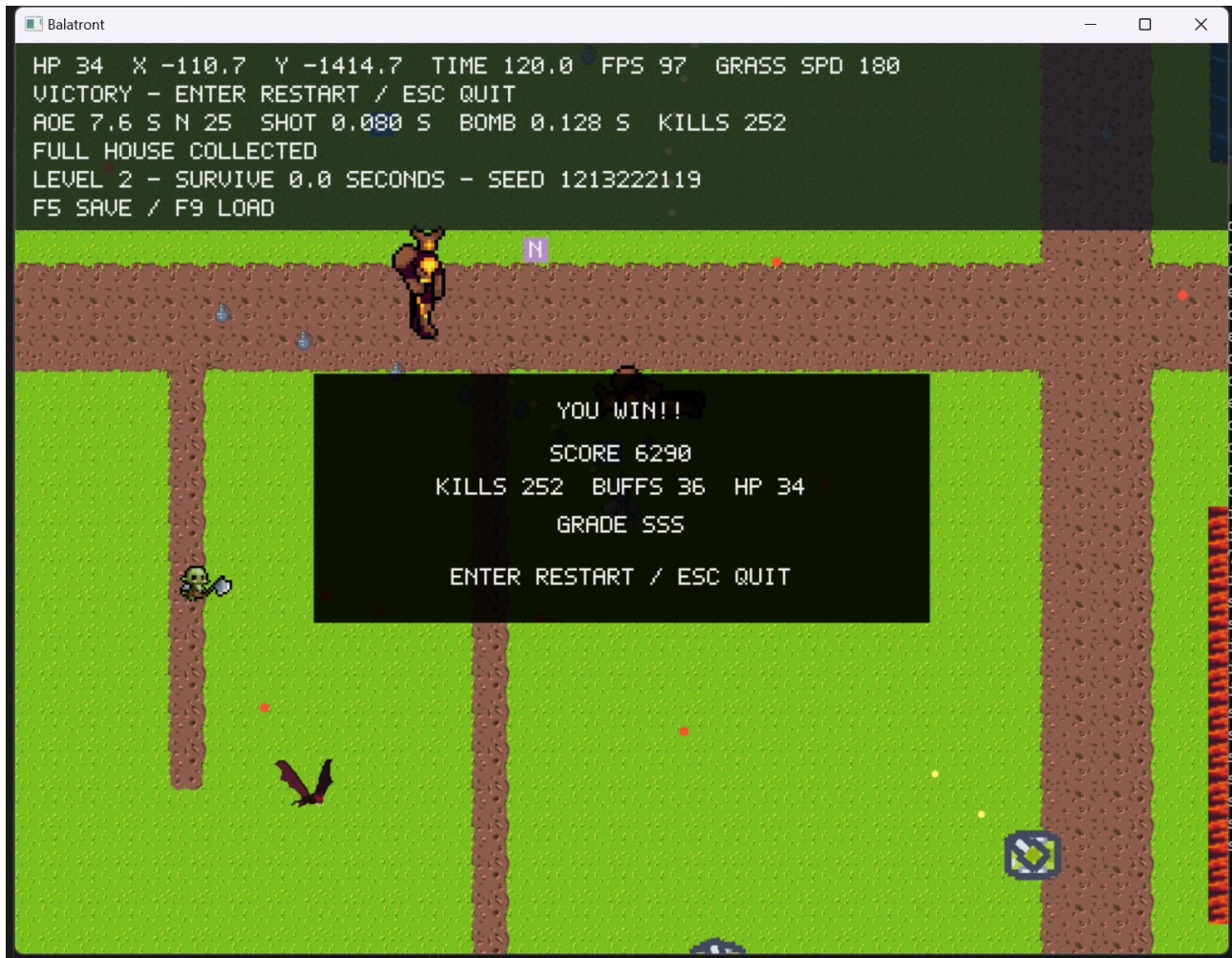


Figure 4 shows the victory screen after Level 2 reaches 120 seconds. This result has 252 kills, 36 upgrades, 34 health, a final score of 6290, and an SSS grade.

8 Save at Any Time and Load from the Same Point

The player can press F5 to save while the game is running or paused, and press F9 to load. The save file records the level number, level time, portal state, player position and previous-frame position, health, invincibility time, upgrades, camera, enemy array, three projectile arrays, dropped items, medkits, attack cooldowns, fixed-map tiles, and the infinite-map seed.

Archive writes integers, floating-point values, Boolean values, and vectors in a fixed order. The start of the file stores an identifier and format version, and the end stores a checksum. During loading, the code checks value ranges, array capacities, level state, and the checksum, so a damaged file is not loaded into the game.

During saving, the game first writes `savegame.tmp`. After writing is finished, the old save is renamed to `savegame.bak`, and then the temporary file is renamed to `savegame.dat`. If writing is interrupted, the original save still has a chance to be kept. After a successful load, the game enters a paused state. The player presses Esc or Enter to continue, which avoids taking an attack immediately on the first frame after loading.

Saving needs to iterate through the fixed-capacity enemy, projectile, and drop arrays, so the complexity is $O(E+P+U)$. The file size has an upper limit and does not keep increasing with the number of saves.

9 Visual Studio Performance Analysis

The following CPU and GPU performance data were collected using an x64 Release build.

The CPU analysis shows that `TileMap::draw` takes 43.40%, `EnemySprites::drawClip` takes 24.06%, and `Game::paintInformation` takes 13.79%. The main CPU cost comes from drawing the map, enemy sprites, and the HUD.

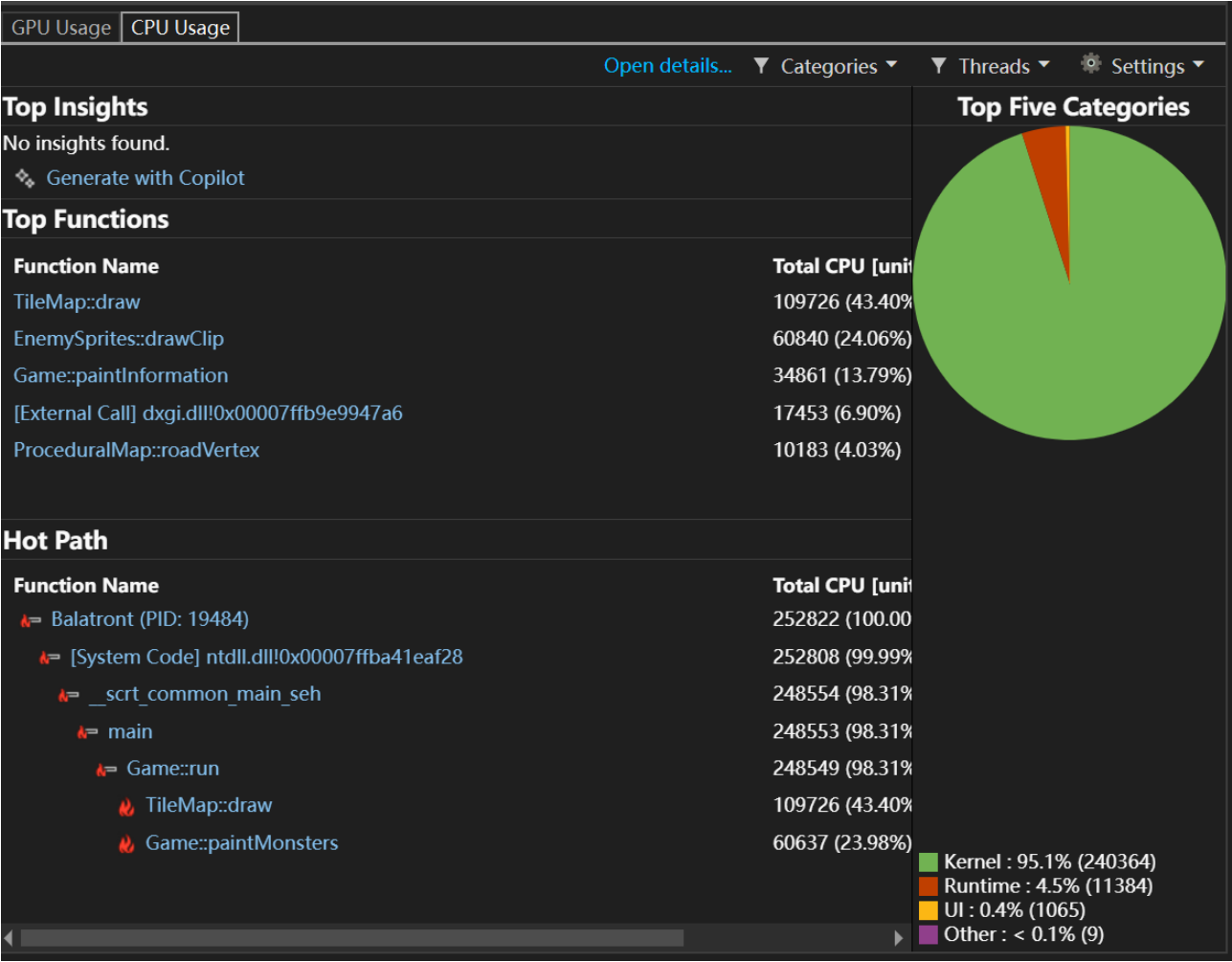


Figure 5. Visual Studio CPU usage analysis result.

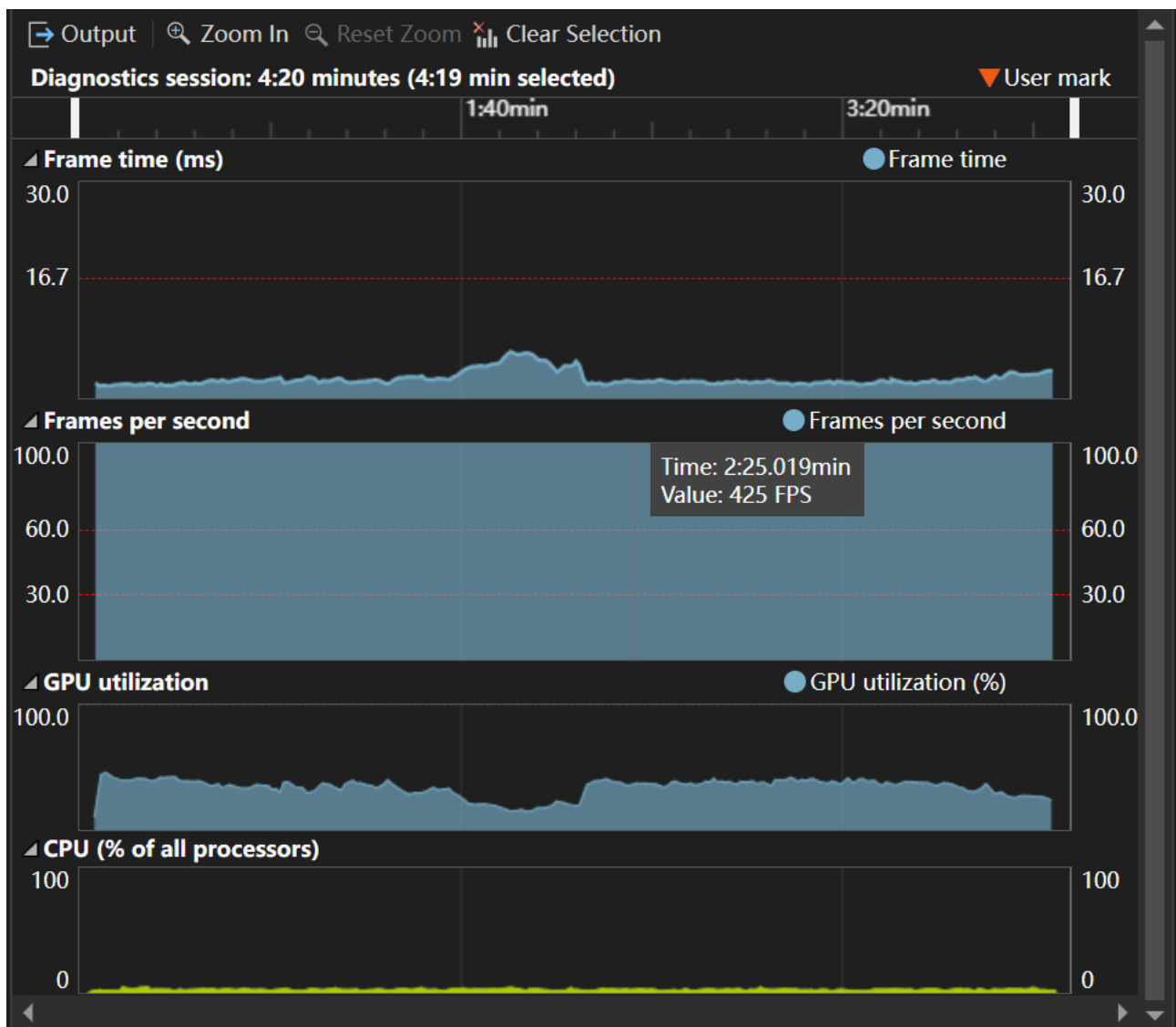


Figure 6. Visual Studio GPU usage analysis result.

10 Project Limitations

The collision system does not use spatial partitioning. Every player projectile may check all enemies, so performance drops clearly when there are many enemies and projectiles. A better solution would be to divide the world into a grid and only check enemies in cells near the projectile. In addition, controller input depends on the Xbox Controller interface provided by the module framework. If a different controller type cannot be recognised by the system as a compatible controller, it may not work. Also, although there are already CPU and GPU analysis results from a complete game run, repeated measurements on the same device and with the same Release build configuration are still missing. The final report still needs to add the computer specification and record average FPS and minimum FPS at fixed time points.

at any time from the same point. The main implementation choices are fixed arrays for controlling entity capacity, continuous circle sweeps for reducing tunnelling of fast objects, text files for driving numerical values and maps, and coordinate hashing for generating an infinite world without saving all world data. The main current limitation is the collision and software-rendering cost caused by large numbers of enemies and projectiles late in the game. Future optimisation can use a spatial grid, sprite caching, and map-region caching.

11 What I Would Do If I Started Again

If I developed this project again, I would first decide the core data structures, resource formats, and level flow, and then add features step by step. The player, enemies, projectiles, and dropped items could still use fixed arrays because their maximum sizes are clear and they are easy to save. However, I would add a simple spatial grid earlier, so collision detection only checks nearby objects from the beginning and the core loop does not need to be changed later.

For rendering, I would first make a resource manager that can cache scaled sprites. Static images and normal animations would be generated at their final sizes when loaded, and drawing would only copy the pixels that need to be shown. Only images that really need rotation, such as the turret barrel, would use per-pixel rotation.

Finally, I would clearly define the save-data version and field ranges in the first version. Resource configuration, map format, and save format would stay independent, and the game code would only read these data. Then, when balance values, art resources, or maps are changed, there is no need to search for hard-coded values across several C++ files.

12 Conclusion

This project completes a 2D twin-stick shooter that can run from the start of the game to either a victory or defeat result. The game includes a finite map and an infinite map, a virtual camera, four NPC types, increasing spawn frequency, direct chasing and turret attacks, complete collisions between the player, enemies, terrain, and projectiles, an eight-direction linear attack, a highest-health AOE attack, two upgrade types, two 120-second levels, FPS and score display, and save/load